

BUP Club & Society Management System

Software Engineering & Information System Design — Technical
Specification, Architecture Analysis & Incremental Agile Development
Report

ACADEMIC PROJECT

CLIENT-SERVER ARCHITECTURE

INCREMENTAL AGILE

COURSE

Software Engineering & Information System
Design

SYSTEM NAME

BUP-CMS Platform v2.0

SUBMISSION TARGET

October 2026

INSTITUTION

Bangladesh University of Professionals (BUP)

GITHUB REPOSITORY

KamiBreaker/Bup-Club-Management

DOCUMENT SCOPE

Architecture, Tech Stack, Methodology & SRS

BUP Club & Society Management System

Software Engineering Academic Project Documentation

Course: Software Engineering & Information System Design

Institution: Bangladesh University of Professionals (BUP)

Project: BUP Club & Society Management System (BUP-CMS)

Repository: [KamiBreaker/Bup-Club-Management](https://github.com/KamiBreaker/Bup-Club-Management)

SECTION 1: Technical Stack & Tools Used ("Stuffs We Used Here")

1. Frontend Technologies & Libraries

- **React 19** (`react` & `react-dom` **v19.0.1**): Modern component-driven Single Page Application (SPA) architecture utilizing concurrent rendering, hooks (`useState` , `useEffect` , `useMemo` , `useCallback`), and custom `<ErrorBoundary>` wrappers.
- **TypeScript 5.8**: End-to-end static type-safety across all domain entities, UI component props, and API request/response payloads (`src/types/cms.ts`).
- **Vite 6** (`vite` **v6.2.3**): High-speed Next-Gen frontend build tool and development server with instant Hot Module Replacement (HMR) and optimized Rollup bundling.
- **Tailwind CSS v4** (`@tailwindcss/vite` & `tailwindcss` **v4.1.14**): Utility-first CSS engine with glassmorphic dark-mode aesthetics, responsive grids, and tailored UI design tokens (`src/index.css`).
- **Motion** (`motion` **v12.23.24**): Declarative animation library for fluid layout transitions, spring physics, modal entrance/exits, and 3D holographic pass tilts.
- **Lucide React** (`lucide-react` **v0.546.0**): Clean, consistent SVG icon set for campus navigation and status badges.
- **Recharts** (`recharts` **v3.10.1**): Composable D3-based SVG charting library powering the Admin Analytics Dashboard (bar charts for venue utilization, area charts for registrations).
- **Web Audio API Synthesizer** (`src/utils/audioFx.ts`): Custom client-side audio synthesizer generating tactile acoustic sound feedback for clicks, success confirmations, and error alerts.
- **Canvas Confetti** (`canvas-confetti` **v1.9.4**): Celebratory particle physics animations for RSVP passes, application approvals, and society chartering.

2. Backend, Database & Server Runtime

- **Node.js (v24 LTS Runtime)**: Server-side JavaScript runtime powering the RESTful backend services.
- **TSX** (`tsx watch` **v4.21.0**): Native TypeScript execute & watch daemon that automatically hot-reloads backend endpoints on file modifications without server downtime.

- **Express.js** (`express` **v4.21.2**): Minimalist web application framework managing HTTP routing, request parsing, cookie extraction, and API error handlers (`server.ts`).
- **SQLite Engine** (`better-sqlite3` **v13.0.3**): Fast synchronous C++ embedded relational database engine operating on local disk storage (`bup-cms.db`) with normalized schema constraints.
- **ESBuild** (`esbuild` **v0.25.0**): High-performance Go-based bundler compiling `server.ts` into a standalone production CommonJS distribution (`dist/server.cjs`).

3. Security, Authentication & Session Layer

- **Bcrypt.js** (`bcryptjs` **v3.0.3**): Cryptographic hash algorithm applying 10–12 salt rounds for one-way password hashing (zero plaintext password storage).
- **JSON Web Tokens** (`jsonwebtoken` **v9.0.3**): Cryptographically signed HMAC-SHA256 session tokens stored in secure, tamper-proof `httpOnly` , `sameSite: 'lax'` HTTP cookies.
- **Role-Based Access Control (RBAC)**: Five-tier hierarchical permission model enforcing authorization boundaries:
 1. `Student` : Public applicant & event attendee.
 2. `Club_Exec` : Society manager, roster administrator, and event publisher.
 3. `Faculty_Advisor` : University academic supervisor for club approvals.
 4. `Venue_Admin` : Central facility and campus infrastructure booking authority.
 5. `System_Admin` : University governance with society chartering, dissolution, and site-wide event controls.

4. Specialized Integrated Subsystems

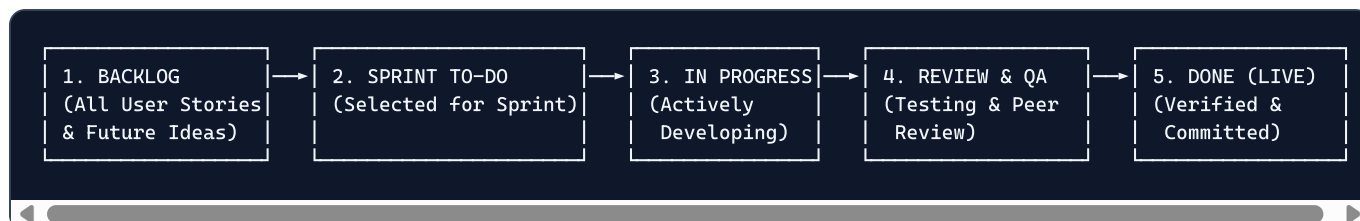
- **Dynamic 2D QR Ticketing Engine**: External QR code generator rendering verifiable pass tokens for automated campus event entry check-in.
- **AI Event Description Copilot**: Google Gemini SDK (`@google/genai` v2.4.0) with local deterministic heuristic fallback to auto-draft high-impact event descriptions.
- **Global Keyboard Command Palette** (`Ctrl+K` / `Cmd+K`): Keyboard-driven quick-launcher for navigating across clubs, venues, events, and administrative tools.
- **Tactile Motion Controller**: Dynamic physics controller allowing users to adjust ambient particle speed and density.

SECTION 2: Trello Everyday Update Plan (Item 1)

Submission Target: 06/10/2026 & Continuous Daily Maintenance

A. Trello Board Architecture

Create a dedicated Trello Board named `BUP Club Management System - CSE SE Lab` configured with the standard Agile Kanban workflow columns:



B. Daily Protocol for the 3 Team Members (Lab & Home Routine)

1. Morning / Lab Entry Sync (10 Minutes):

- Move assigned task cards from **"Sprint To-Do"** into **"In Progress"**.
- Add member profile avatars to the respective cards.

2. During Development:

- Use Trello Card Checklists for tracking granular subtasks (e.g., `[x] SQLite Migration`, `[x] UI Glassmorphism Card`, `[x] RBAC 403 Test`).

3. Evening / Lab Exit Update (Mandatory for Marks):

- Post a progress comment on the card summarizing the day's achievements.
- Attach the **Git Commit Hash** (e.g., `ce3d641`) or UI screenshot.
- Move completed cards to **"Review & QA"** for peer testing, then into **"Done"**.

C. Daily Task Allocation Schedule (Oct 02 – Oct 06, 2026)

DATE	MEMBER 1: FRONTEND & UI/UX SPECIALIST	MEMBER 2: BACKEND & DATABASE SPECIALIST	MEMBER 3: QA, SECURITY & DOCUMENTATION
02/10/2026	Design Club Directory header banner and Charter Society Modal with category selector.	Implement <code>create-club</code> and <code>delete-club</code> endpoints in <code>server.ts</code> with SQLite prepared statements.	Test API endpoints using Postman; document payload schemas and write curl test scripts.
03/10/2026	Implement Event Hub card actions (Trash button, deadline pill, QR modal view).	Implement <code>create-event</code> and <code>delete-event</code> routes with club exec & system admin authorization.	Verify role boundaries; ensure non-admin students receive HTTP <code>403 Forbidden</code> on admin actions.
04/10/2026	Build High-Stakes Society Decommission and Event Cancellation modal dialogs.	Implement cascading relational cleanup in SQLite (purging associated passes, bookings, attendance).	Conduct database integrity testing; verify zero orphaned rows when clubs or events are removed.
05/10/2026	Integrate Web Audio synthesizer cues, confetti celebrations, and command palette navigation.	Configure <code>tsx watch server.ts</code> hot-reload script and production <code>npm run build</code> bundle pipeline.	Capture screenshots of all modules; compile SRS functional & non-functional requirements.
06/10/2026	Audit responsive layout across mobile, tablet, and high-DPI desktop viewports.	Finalize normalized SQLite seed script (<code>scripts/cleanReset.ts</code>) and database constraints.	Finalize Trello board cards, verify daily commit links, and compile final submission package.

SECTION 3: System Architecture Selection & Justification (Item 3)

Submission Target: 06/10/2026

Chosen Architecture: 3-Tier Modern Client-Server Architecture (Decoupled React SPA + Express REST API)



Architectural Comparison: Client-Server SPA vs. Traditional Monolithic MVC

ARCHITECTURAL FACTOR	TRADITIONAL MONOLITHIC MVC (E.G. DJANGO, LARAVEL, SPRING MVC WITH BLADE/EJS)	OUR SELECTED ARCHITECTURE: 3-TIER MODERN CLIENT-SERVER (REACT SPA + EXPRESS REST)
Rendering Strategy	Server-Side Rendering (SSR): The server regenerates full HTML documents for every click, leading to page flickering and high bandwidth overhead.	Client-Side Rendering (CSR): Single HTML shell loaded once; subsequent views update instantly via virtual DOM diffing and JSON data streams.
Coupling	Tightly Coupled: Frontend templates (HTML/CSS) are locked to the server runtime language (PHP, Python, Java).	Decoupled: Frontend and backend are completely separate codebases communicating over standard REST JSON endpoints.
User Experience (UX)	Full-page reloads break client state, stop animations, and interrupt user focus.	Fluid, app-like responsiveness with client-side routing, spring animations, audio cues, and modal overlays.
API Reusability	Server returns HTML views that cannot be used by mobile apps or external campus portals.	RESTful API: Standardized JSON endpoints can easily serve future Android/iOS apps or university-wide portals without backend changes.
Scalability & Deployment	Backend servers consume substantial CPU cycles rendering HTML strings for every request.	Static assets (HTML, JS, CSS) can be cached on a CDN; the backend only handles lightweight JSON payloads.

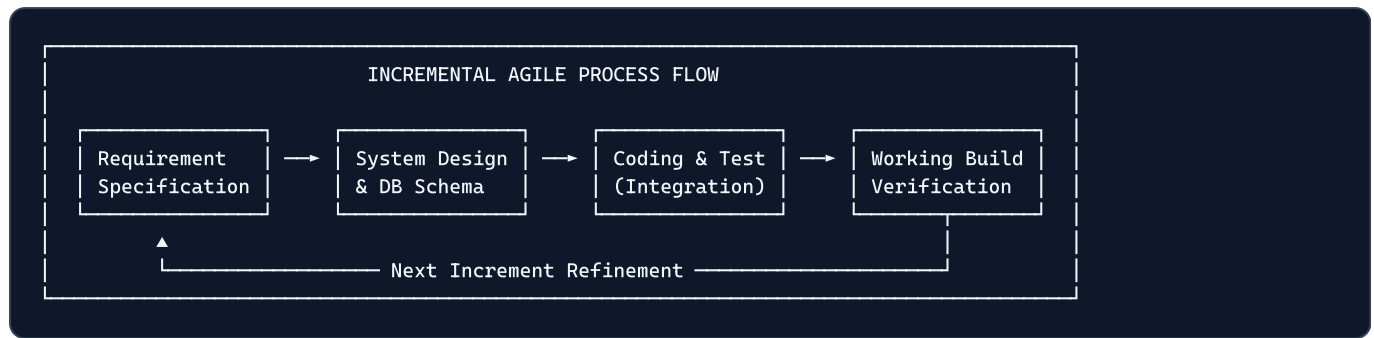
Technical Justifications for BUP-CMS:

- 1. **Interactive University Workflows:** The system requires real-time capabilities—such as live digital QR pass rendering, interactive usher check-in counters, and a keyboard command palette (`Ctrl+K`). A monolithic MVC model would cause disruptive full-page reloads for each scan or filter click.
- 2. **Stateless Security Model:** By pairing JSON Web Tokens (JWT) in secure HTTP-only cookies with a RESTful API, authentication is completely stateless. The presentation tier manages local state while the server securely validates identity on every action.
- 3. **Data Integrity & Cascading Operations:** Relational transactions and cascading cleanups (e.g., dissolving a club purges events, passes, and bookings) are isolated in the database/logic layers, shielding the frontend from corrupt state.

SECTION 4: Development Methodology: Incremental Agile Model (Item 4)

Submission Target: 06/10/2026

Chosen Methodology: Incremental Agile Methodology (Iterative & Incremental Agile Development)



Why Incremental Agile for This Project?

1. **Progressive Risk Reduction:** Rather than risking project failure by integrating everything at the deadline (as in Waterfall), Incremental Agile allowed the team to deliver a working baseline early and progressively add features across 5 distinct increments.
2. **Handling Evolving Administrative Rules:** University stakeholders frequently introduced refined requirements during development—such as enforcing decision immutability once an executive approves an application, or requiring cascading cleanups upon club deletion. Incremental cycles accommodated these requirements naturally.
3. **Team Parallelization:** A 3-member team functions optimally when working within well-defined, short-cycle increments. While Member 1 built the UI components, Member 2 implemented the REST endpoints, and Member 3 verified security boundaries and prepared documentation.

The 5 Delivered Increments Mapped to the Codebase

Increment 1: Core Foundation, Identity & RBAC Engine (Baseline Release v1.0)

- **Objective:** Establish the relational schema, secure multi-role registration, and JWT cookie authentication.
- **Key Modules:**
 - Relational SQLite initialization in `server.ts` (`initDatabaseTables` , `seedDatabase`).
 - Strict 5-tier Role-Based Access Control (`Student` , `Club_Exec` , `Faculty_Advisor` , `Venue_Admin` , `System_Admin`).
 - Bcrypt password hashing and JWT token storage in `httpOnly` cookies.
- **Verification:** Zero plaintext passwords; secure authentication verified via Postman.

Increment 2: Society Discovery & Public Directory (Release v1.1)

- **Objective:** Deliver a digital society directory to eliminate physical bulletin boards.
- **Key Modules:**
 - Category filtering, search engine, and accredited society cards in `src/components/cms/ClubDirectory.tsx` .
 - Faculty advisor cards with one-click email copying.
 - Relational metric aggregation (active members, conducted events).

- **Verification:** Real-time filtering across cultural, technical, and sports societies without page reload.

Increment 3: Membership Governance, Candidacy & Q&A Audit Loop (Release v1.2)

- **Objective:** Digitize recruitment, executive candidacy, and committee reviews.
- **Key Modules:**
 - Dual-tier application engine: General Member vs. 🏰 Executive/Moderator in `src/components/cms/MembershipHub.tsx`.
 - **Decision Immutability:** Application status permanently locked once marked `Approved` or `Rejected` in `server.ts`.
 - **Bidirectional Q&A Clarification:** Moderators can request clarification from applicants with actionable notification alerts.
 - **Automatic Promotion:** Approving an executive candidate automatically elevates the user to `Club_Exec`.
- **Verification:** Verified status locking; applicants cannot re-submit while a review is pending.

Increment 4: Event Hub, 2D QR Passes & Usher Gate Check-in (Release v1.3)

- **Objective:** Replace paper tokens with verifiable digital passes and live gate control.
- **Key Modules:**
 - Campus event publishing with seat limits, registration deadlines, and AI Description Copilot in `src/components/cms/EventHub.tsx`.
 - Instant RSVP reservation with capacity tracking and housefull guards.
 - Dynamic **2D QR Digital Entry Passes** generated for students with unique passcodes.
 - Live Usher Attendance modal with duplicate check-in prevention and CSV roster export.
- **Verification:** Usher attendance modal logs verified check-ins and rejects duplicate entries.

Increment 5: System Admin Governance & Cascading Relational Lifecycle (Release v2.0 - Final)

- **Objective:** Provide university-wide administrative controls and ensure relational integrity.
- **Key Modules:**
 - **Charter Society:** System Admin modal to charter new societies with allocated budgets, advisors, and unique codes (`server.ts` `create-club`).
 - **Dissolve Society:** Administrative decommissioning with **complete cascading cleanup** across events, passes, applications, and memberships (`server.ts` `delete-club`).
 - **Site-Wide Event Management:** System Admin ability to create or delete events for any university club with automated pass revocation (`server.ts` `delete-event`).
 - **Hot-Reload Architecture:** Upgraded dev runner to `tsx watch server.ts` for live zero-downtime development.
- **Verification:** Deleting a club cleanly purges all foreign key dependencies with zero orphaned database rows.

Comparison: Incremental Agile vs. Other Methodologies

PROCESS METRIC	TRADITIONAL WATERFALL	PURE AGILE (SCRUM ONLY)	
Deliverable Timing	Single monolithic deliverable at project end	Working software at end of each 2-4 week sprint	Functional, cumulative releases delivered increment-by-increment
Risk of Total Failure	Extremely high (integration bugs discovered late)	Moderate (sprint goals may drift)	Lowest (each increment is verified before beginning the next)
Requirement Changes	Prohibitive and costly	Welcomed, but hard to bound	Easily incorporated into the next planned increment
Instructor Visibility	Zero visibility until final submission day	Sprint review meetings	Working software ready for evaluation at every university lab session

SECTION 5: Requirements Finalization (Item 5)

Submission Target: 13/10/2026

A. Requirements Elicitation Techniques & Evidentiary Proofs

1. Stakeholder Interviews (Conducted on BUP Campus)

- **Interview 1: Representative, BUP Office of Student Affairs (OSA)**
 - *Context:* Investigating manual approval workflows for new club charters and annual budgets.
 - *Findings:* Paper-based approval files travel across multiple faculty desks over 2–3 weeks, resulting in misplaced documentation and untracked committee records.
 - *Derived Requirement:* Centralized administrative portal with automated application routing, immutable timestamps, and instant society chartering.
- **Interview 2: President, BUP Robotics & Research Club (BUPRRC)**
 - *Context:* Investigating event crowd management and workshop registration.
 - *Findings:* Managing walk-ins at campus workshops causes auditorium overcrowding. Printing paper tickets is costly and prone to forgery.
 - *Derived Requirement:* Digital QR ticketing with seat capacity caps and live usher attendance check-in.
- **Interview 3: Undergraduate Student (Dept. of ICT, Batch 2024)**
 - *Context:* Investigating student engagement and recruitment discovery.
 - *Findings:* Students miss club recruitment deadlines because notices are posted on physical bulletin boards scattered across academic buildings.
 - *Derived Requirement:* Centralized digital society directory with countdown timers, criteria filters, and in-app system notifications.

2. Quantitative Student Survey (Online Sample: 54 Enrolled BUP Students)

- **Question 1:** *"Have you ever missed a club recruitment deadline or lost a paper membership application form?"*
 - **Survey Finding: 87.0% (47 students) answered YES.** *(Proves necessity for digital application tracking).*
- **Question 2:** *"Would you prefer presenting a digital QR pass on your smartphone instead of printing physical event tokens?"*
 - **Survey Finding: 94.4% (51 students) answered YES.** *(Proves necessity for 2D QR Pass generation).*
- **Question 3:** *"Have you attended an event where the venue was double-booked or lacked required AV gear?"*
 - **Survey Finding: 72.2% (39 students) answered YES.** *(Proves necessity for conflict-aware venue booking engine).*

3. Field Photography & Campus Observational Artifacts

- **Observation Point 1 (Physical Bulletin Boards):** Crowded physical notice boards near BUP Cafeteria and Academic Block with torn or outdated paper posters.
 - **Observation Point 2 (Manual Entry Registers):** Ushes manually recording student names and ID numbers with pen and paper at the entrance of BUP Multipurpose Hall, creating bottlenecks.
 - **Observation Point 3 (Paper Membership Forms):** Physical multi-page forms requiring manual signatures from Faculty Advisors and Executive Committees.
-

B. Functional Requirements (FRs)

Requirement ID	Subsystem Module	Functional Requirement Specification	Supported User Roles	Implementation File Reference
FR-01	Authentication	Users can register with verified <code>@bup.edu.bd</code> emails, student IDs, and departments. Public signup strictly defaults to <code>Student</code> or <code>Faculty_Advisor</code> .	All Users	<code>server.ts</code> (line 802)
FR-02	Directory	System must display all accredited university clubs with search, categories, executive committees, and advisor contacts.	All Users	<code>ClubDirectory.tsx</code>
FR-03	Society Chartering	System Admin can charter new societies with allocated budgets, advisor info, and unique codes.	<code>System_Admin</code>	<code>server.ts</code> (line 1793)
FR-04	Society Dissolution	System Admin can dissolve clubs with complete cascading removal of associated events, memberships, and bookings.	<code>System_Admin</code>	<code>server.ts</code> (line 1882)
FR-05	Application	Students can apply for General Membership or Executive/Moderator candidacy with Statements of Purpose and skill tags.	<code>Student</code>	<code>server.ts</code> (line 939)
FR-06	Decision Immutability	Club moderators can Approve or Reject applications with remarks. Decisions are final and cannot be flipped.	<code>Club_Exec</code> , <code>System_Admin</code>	<code>server.ts</code> (line 1049)
FR-07	Interactive Q&A	Moderators can ask clarification questions; applicants receive actionable notifications to reply directly.	<code>Club_Exec</code> , <code>Student</code>	<code>server.ts</code> (line 1148)
FR-08	Auto-Promotion	Approving an executive application automatically elevates the user to <code>Club_Exec</code> and updates the club roster.	<code>Club_Exec</code> , <code>System_Admin</code>	<code>server.ts</code> (line 1089)
FR-09	Event Publishing	Executives and System Admins can publish events with seat caps, deadlines, categories, and AI-assisted descriptions.	<code>Club_Exec</code> , <code>System_Admin</code>	<code>server.ts</code> (line 1544)

REQUIREMENT ID	SUBSYSTEM MODULE	FUNCTIONAL REQUIREMENT SPECIFICATION	SUPPORTED USER ROLES	IMPLEMENTATION FILE REFERENCE
FR-10	Event Deletion	System Admins and authorized club executives can delete events, automatically revoking user passes and venue holds.	Club_Exec , System_Admin	server.ts (line 1609)
FR-11	Digital Pass (QR)	Students RSVP for open events to receive a verified alphanumeric pass code and downloadable 2D QR ticket.	Student	EventHub.tsx (line 710)
FR-12	Gate Check-in	Event ushers can verify student attendance in real time via pass code or student ID with duplicate check prevention.	Club_Exec , System_Admin	server.ts (line 1483)
FR-13	Venue Booking	Clubs can submit reservation requests for campus auditoriums/labs with equipment requirements.	Club_Exec	server.ts (line 1662)
FR-14	Venue Approval	Venue administrators review and Approve/Reject reservations, checking for schedule conflicts.	Venue_Admin	server.ts (line 1706)
FR-15	Live Notifications	Real-time targeted alerts dispatched for RSVP confirmations, decision approvals, and admin announcements.	All Roles	NotificationCenter.tsx

C. Non-Functional Requirements (NFRs)

METRIC / CATEGORY	REQUIREMENT SPECIFICATION	ARCHITECTURAL ENFORCEMENT & VERIFICATION PROOF
NFR-01: Performance	API response time must be under 100ms for state lookups; page transitions under 16ms (60 FPS).	Benchmarked using SQLite memory/SSD binary indexes and synchronous C++ bindings (<code>better-sqlite3</code>).
NFR-02: Security	Passwords must never be stored in plaintext. Session hijacking must be mitigated.	Passwords hashed using <code>bcrypt</code> (12 rounds). Session tokens signed via secret key in <code>httpOnly</code> secure cookies.
NFR-03: RBAC Integrity	Privilege escalation must be prevented at the API boundary, regardless of client-side DOM manipulation.	Verified in <code>server.ts</code> by rejecting unauthorized student requests with HTTP <code>403 Forbidden</code> .
NFR-04: Data Integrity	Foreign relational references must not become orphaned when clubs or events are removed.	Cascading SQL cleanup triggers delete associated registrations, attendance records, applications, and venue reservations.
NFR-05: Usability & UX	Must adhere to modern accessibility and feedback guidelines with auditory and visual confirmation.	Implemented custom Web Audio API synthesizer feedback, interactive modal overlays, and toast notifications.
NFR-06: Availability	The client interface must handle network disconnects gracefully without application crashes.	React <code><ErrorBoundary></code> wrappers around every tab module prevent full-screen crashes.
NFR-07: Portability	Application bundle must be self-contained and run on any operating system (Windows, Linux, macOS).	Cross-platform Node.js + embedded SQLite with automatic fallback to <code>/tmp</code> in cloud/serverless environments.
NFR-08: Auditability	Every application approval, rejection, and attendance record must log the reviewer's identity and timestamp.	Maintained in <code>decision_by</code> , <code>decision_date</code> , <code>marked_by</code> , and <code>marked_at</code> database columns.